When Does Importance-Aware Tiering of Multimodal KV Caches Pay Off? A Hypothesis-Driven Simulation Study on Real Vision-Language Attention

Sudarshan Sudhakar

Independent research. Code, traces and all result tables: `KV-Tier` repository

Abstract

Vision-language models (VLMs) turn each image into hundreds of visual tokens, and the key–value (KV) cache they create dominates memory traffic during decoding. Algorithmic work compresses this cache but assumes one flat memory. We ask the *architecture* question instead: should KV entries of different importance live in different memory tiers, and what does moving an entry between tiers cost? We model a two-tier system, a conventional FP16 HOT tier and a compute-in-memory INT4 WARM tier, driven by a live per-entry importance estimate and a hysteretic promote/demote policy that pays an explicit reconfiguration cost. The study is a chain of seven experiments in which each result motivates the next. Policy gains found on synthetic attention *did not replicate* on real SmolVLM attention; the effect appeared only when entries were modelled per (token, layer); a layer/role prior was tested as the explanation and falsified; and the effect was then established on 42 images (natural photos, charts, documents) and two model sizes under a rule fixed in advance. With dense reads and a 10× reconfiguration cost, the dynamic policy lowers energy by **22.3%** (95% CI 21.9–22.8) on SmolVLM-256M and **23.4%** on SmolVLM-500M relative to static modality tiering, at neutral quality, and thresholds transfer across model sizes unchanged. The saving survives a reconfiguration cost up to about 20× and breaks even near 50×. It vanishes under sparse (top-10%) attention reads. Two checks address the strongest objections to a simulation study. The saving is robust to a degraded importance signal (noise, stale, or sampled every 8th step: −18.6 to −21.8%) but not to collapsing it to one score per token (−7.6%). With INT4 actually applied inside SmolVLM-256M, the dynamic placement changes the next-token distribution **21.8% less** than static modality (KL, lower on 37 of 42 images) and **9.4× less** than random placement of the same number of INT4 entries.

Keywords: KV cache · vision-language models · tiered memory · compute-in-memory · memory-centric architecture · negative results.

1 Introduction

A vision-language model prefills an image as hundreds to thousands of visual tokens. Each token adds a KV entry in every transformer layer, and during autoregressive decoding every new token reads that cache again. For long multimodal contexts the cache, not the arithmetic, sets the energy and bandwidth bill. A large body of recent work shrinks the cache by pruning, quantizing or merging visual tokens [1, 2, 3, 4, 5]. All of it assumes a flat, undifferentiated memory and decides only *what to discard*.

We study the complementary architectural question. If entries differ in how much attention they receive, should they also differ in *where they live*? We consider a memory with a HOT tier (conventional DRAM/HBM, FP16, lossless) and a WARM tier (compute-in-memory, INT4, lower energy per byte, slower per access, lossy). Placing a lightly attended entry in WARM saves energy; placing a heavily attended one there costs quality. Importance changes as decoding proceeds, so a policy may *move* entries, but every move is paid for. The design question is whether live, per-entry placement beats the simple static rule "visual tokens in WARM, text in HOT", and under which conditions.

Approach. We build a SimPy simulator with an explicit cost model for access, quantization error and tier moves, and we feed it *real* decode-time attention extracted from SmolVLM. We treat the work as a sequence of falsifiable hypotheses. Each experiment is motivated by the weakness of the previous one (Fig. 1). Selection rules and success criteria were written in the lab notes before the corresponding runs (a self-imposed discipline, not an external registration), and every deviation from them is logged.

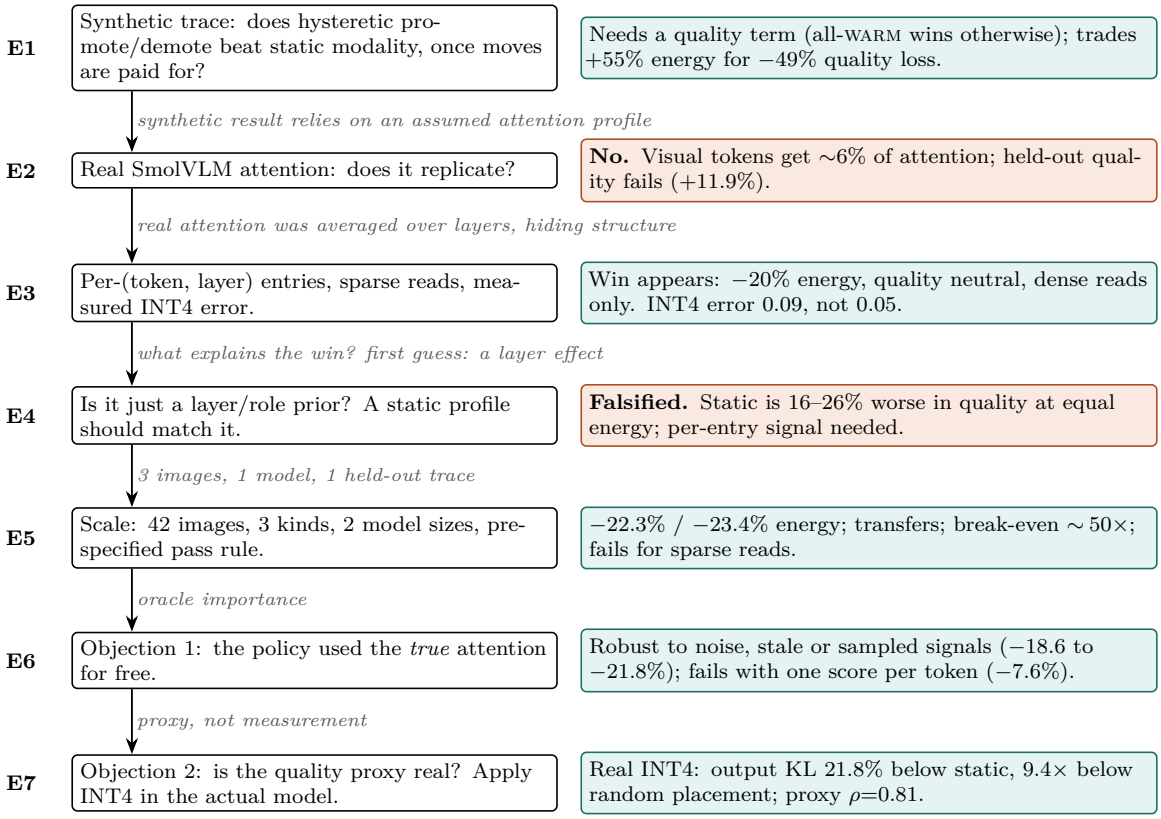


Figure 1: The study as a chain of hypotheses. Each box states the question, the finding, and (on the arrow) the weakness that forced the next experiment. Red marks the two results that overturned an earlier belief.

Contributions.

1. A reconfiguration-aware, quality-aware simulator for modality- and importance-aware KV placement, with a measured (not assumed) INT4 error and a linearity property that makes reconfiguration cost an exact post-hoc parameter.
2. A negative result with a diagnosis: the synthetic-trace win does not replicate on real attention, because real visual tokens receive only ∼6% of attention mass; the win appears at per-(token, layer) granularity and is *not* explained by a layer or role prior (a static profile is 16–26% worse in quality at matched energy).
3. A pre-specified scale-up (42 images, three image kinds, two model sizes, leave-one-trace-out selection, bootstrap CIs) establishing **−22 to −23% energy at neutral quality** for dense reads, with a cost break-even near 50× and a clear failure region (sparse reads).
4. Two checks aimed at the strongest objections to a simulation study: robustness to a *non-oracle* importance signal (Sec. 5.6) and agreement of the quality proxy with *measured* changes in the real model's output distribution when INT4 is actually applied (Sec. 5.7).

2 Related Work

Algorithm-level visual KV compression. VL-Cache [1] (modality- and sparsity-aware budgeting), ZipVL [2] (dynamic token sparsification), AirCache [3] and TopV [4] (inter-modal relevance and token pruning) and FastV [5] (early-layer visual token pruning) show that visual tokens are, on average, less important to decoding than text tokens and can be compressed with small accuracy loss. For text LLMs, H2O [6] and SnapKV [7] keep heavy-hitter entries, StreamingLLM [8] keeps attention sinks plus a recent window, KIVI [9] quantizes the cache to 2 bits, and Quest [10] reads only query-relevant pages. These methods *discard or shrink*. We instead ask where an entry should reside and what it costs to move it, a question they do not pose because their memory is flat.

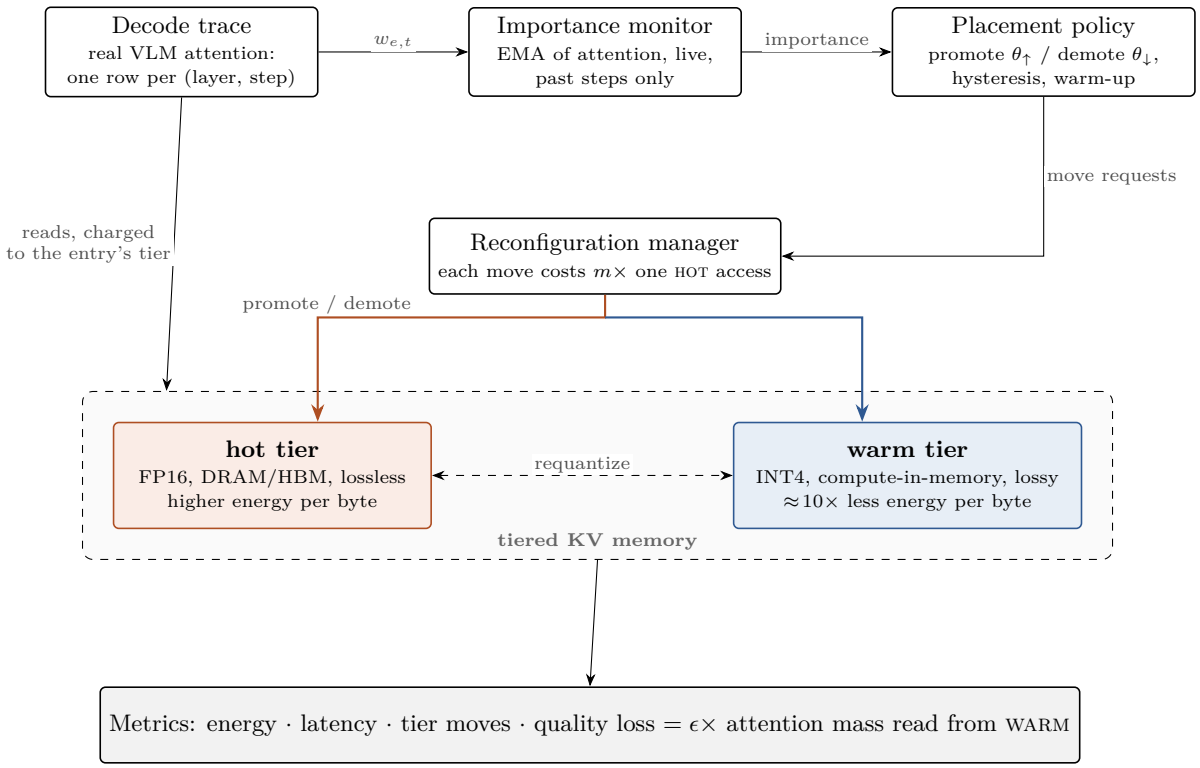


Figure 2: System model. A decode trace supplies, per step, the attention each entry receives. Reads are charged to the entry's current tier. The live importance monitor sees only past weights, never the future. The policy may request a tier change, which the reconfiguration manager charges at a multiple of a HOT access. Quality is accounted separately from energy and latency.

Table 1: Tier cost model (stylised, one shared set of ratios for all policies). The per-access cost is $c_0 + c_1 \cdot$ bytes at the tier's own precision.

Tier	Precision	Base lat. (ns)	Lat./byte (ns)	Base energy (pJ)	Energy/byte (pJ)
HOT	FP16 (2 B/elem)	20	0.010	5.0	8.0
WARM	INT4 (0.5 B/elem)	30	0.015	0.5	0.8

WARM is 10× cheaper per byte in energy, 1.5× slower per access, and stores 4× fewer bytes. Quality weight of a WARM read: INT4 relative error ϵ.

Memory systems for LLM serving. PagedAttention [11] manages KV memory in fixed blocks, and a growing set of near-/in-memory accelerators target attention (single-tier, not modality-aware). Our contribution is orthogonal: a reconfiguration-cost model for *moving* entries between a high-fidelity tier and a low-energy lossy tier, evaluated on real attention, with the conditions under which it fails.

Positioning. Our access-cost constants are stylised ratios (Table 1), not a specific device, so absolute energies are not claims about hardware. What the study can support are *ratios between policies under one shared cost model* and the sensitivity of those ratios to the assumptions that matter.

3 System Model

3.1 Entities and tiers

A decode session has a prefill of n_p tokens (visual or text) followed by T=64 generated tokens. Following the per-layer view of Sec. 5.3, one *entry* is the KV pair of one (token, layer); a model with ℓ layers has ℓ entries per token. Every decode step, every active entry is read once (dense reads) with attention weight $w_{e,t}$, $\sum_e w_{e,t}$=1.

3.2 Quality proxy

Reading entry e from WARM perturbs its key and value by INT4 quantization. We charge a quality loss of $w_{e,t} \cdot \epsilon$, so the per-step quality loss is ϵ times the attention mass served from WARM. The simulator's default is ϵ=0.05, an assumption made before any measurement. We later measured ϵ on real SmolVLM KV tensors (per-token asymmetric min-max INT4, groups of 32): relative L_2 error 0.102 (K) and 0.081 (V), about 0.09 on average and $\sim 1.8\times$ our default; visual and text tokens differ by under 1%. Because loss is linear in ϵ, *every quality ratio between policies is unchanged*, and only absolute quality values scale. Section 5.7 tests whether this proxy actually predicts the real model's behaviour.

3.3 Importance monitor and policy

The monitor keeps an exponential moving average (EMA, decay 0.9) of the attention each entry receives, updated at the end of every step from that step's weights only. The *dynamic hybrid* policy starts each entry at its modality prior (visual → WARM, text → HOT) and then applies a hysteresis rule: promote WARM→HOT when importance exceeds $\theta_\uparrow$ and demote HOT→WARM when it falls below $\theta_\downarrow < \theta_\uparrow$. A warm-up of two EMA updates prevents moves driven by the zero-initialised EMA. Thresholds are expressed in multiples of *uniform attention* $1/n_p$ (so a threshold of 8× means "eight times what a uniform cache would give each entry"), which makes them comparable across traces of different length.

3.4 Reconfiguration and its linearity

A tier change costs m times one HOT access, in both latency and energy (m=10 by default). Policy decisions depend on the importance EMA and never on cost, so a run's totals decompose exactly as

$$E(m) = E_{\text{access}} + m\, E^{(1\times)}_{\text{reconf}}, \tag{1}$$

which is verified by a unit test. We therefore simulate once at m=1 and obtain any other cost as an exact post-hoc sum. This is what makes the cost sweeps in Sec. 5.5 cheap and exact (thresholds are, however, *re-selected* at each cost).

3.5 Baselines and metrics

Flat: everything in HOT. **Static modality**: visual in WARM, text in HOT, never moved; this is the strong baseline throughout and all results are percentages *relative to it*, per trace. **Static layer/role profile**: groups of (modality or role) × layer placed in HOT when their mean importance exceeds a threshold, fixed offline (Sec. 5.4). We report total energy, serialised latency, the quality proxy, and the number of tier moves. **Sparse reads** (Sec. 5.3) read only the top-k% entries by live importance plus entries created in the last step.

4 Evaluation Protocol

Because the policy has free thresholds, the main risk is fitting them to the test set. We adopted three rules and kept them across milestones.

1. **Selection rule.** Thresholds $(\theta_\uparrow, \theta_\downarrow)$ are the pair with the lowest mean energy among those whose mean quality loss is no worse than static modality's on the *training* traces (fallback: lowest quality loss).
2. **Held-out evaluation.** Milestone 5–6: fit on two traces, test on the third. Milestone 7–8: *leave-one-trace-out* over all 42 traces. Results are held-out means.
3. **A fixed pass criterion for the headline claim** (written before the 42-image runs): mean held-out energy saving $\geq$ 10%; mean quality-loss change $\leq$ +2%; paired-bootstrap 95% CI of the energy saving excludes 0; the sign holds in each of the three image kinds. Anything weaker is reported as "not established".

Deviations from pre-set plans are stated where they occur: the choice of damping rule (Sec. 5.1), the change of thresholds to multiples of uniform attention after the first real-data failure (Sec. 5.2), and the widening of the threshold grid after every fold chose the grid corner (Sec. 5.5).

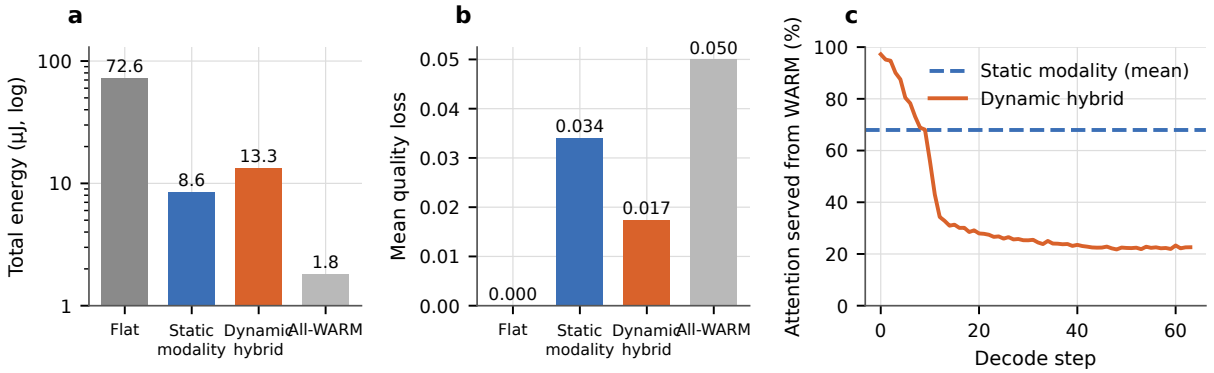


Figure 3: Synthetic trace, default configuration, 10× reconfiguration cost. (a, b) The dynamic policy sits between static modality and flat on energy and halves the quality loss. (c) It does so by promoting the salient visual tokens in the first ∼12 steps, cutting the attention mass served from WARM from ∼97% to ∼36% by step 12 and ∼25% by the end.

Data. Three locally generated images (chart, scene, noise+label) for Milestones 5–6; 42 images for Milestone 7–8 (24 COCO val2017 photos [13], 10 matplotlib charts, 8 rendered text documents), each with a task prompt. Models: SmolVLM-256M-Instruct (primary) and SmolVLM-500M-Instruct [12]. Per image, we greedy-decode 64 tokens and record, at every step, the head-averaged attention row of every layer (30 layers for 256M). The generated text was checked to confirm the model uses the image.

5 Experiments, in the Order They Built on One Another

5.1 E1 — Synthetic traces: is dynamic placement worth its reconfiguration cost?

Question. Does a hysteretic promote/demote policy beat static modality tiering once moves are paid for? **Setup.** A synthetic trace (20 text + 500 visual tokens, 64 steps, a salient 8% of visual tokens, recency bias) reproduces the literature's pattern that visual tokens are on average less important but a salient minority persists.

Step 1: an energy-and-latency-only comparison is degenerate. A trivial *all-WARM* policy beat every dynamic configuration by −78.6% energy at +2.6% latency. With no cost for INT4 error, HOT has no reason to exist. We added the quality term of Sec. 3.2.

Step 2: the policy trades energy for quality. With quality included, the first dynamic policy *lost* to static modality (+67% energy): recency-driven importance spikes at the prefill/decode boundary promoted entries that fell back below the demote threshold within ∼20 steps, so entries round-tripped and each round trip cost 10× a HOT access. Adding a 2-step warm-up (selected over alternatives by quality gain per unit of energy, a post-hoc choice we record as such) cut energy overhead from +64.8% to +55.8% and improved quality from −46.1% to −49.3%; a longer dwell time only batched all moves into one burst.

Finding. On the synthetic trace, dynamic placement buys a large *quality* gain (−48.8% quality loss) for +55% energy over static modality (Fig. 3), a far better rate than interpolating between static and flat (flat: −100% quality loss for +748% energy). **Limitation that drives E2.** This depends on 68% of attention mass landing on WARM-resident salient visual tokens, an assumption of the generator. Whether real VLMs behave this way was untested.

5.2 E2 — Real attention: does the synthetic result replicate?

Question. Replace the generator by SmolVLM-256M's measured decode attention (3 images, layer- and head-averaged, one entry per token). **Finding: it does not replicate.** Visual tokens receive only ∼6% of the attention mass (5–7% across the three images), versus 68% served from WARM in the synthetic trace. A single attention sink at position 0 takes ∼35%, and recently generated tokens another ∼36% (Fig. 4a). The visual attention that exists is heavy-tailed and persistent as assumed (the top 8% of visual tokens hold 61–70% of visual mass, with a 74–83% overlap between the first and last step), but the stakes are small:

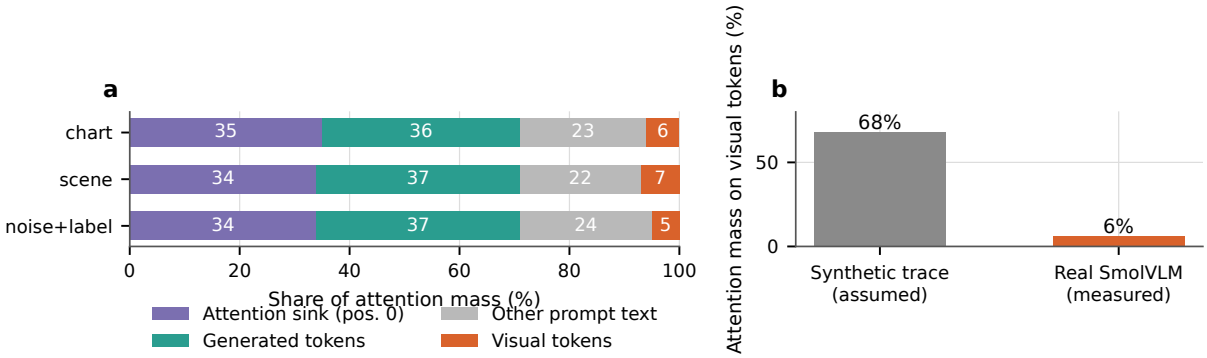


Figure 4: (a) Where real decode attention goes (SmolVLM-256M, three images; layer- and head-averaged). (b) The synthetic trace badly overstated how much attention the WARM tier would serve.

Table 2: E3: held-out trace (fit on the other two), dynamic vs static modality, 10× reconfiguration cost. "Layer-avg" is E2's single entry per token.

Entries / reads	Flat/static energy	Chosen $\theta_\uparrow/\theta_\downarrow$	Energy	Quality loss
Layer-avg, dense	9.3×	2.0/0.4	−14.4%	+11.9%
Layer-avg, top-10%	1.6×	8.0/0.02	+0.2% (no moves)	0%
Per-layer, dense	9.3×	4.0/0.4	**−20.0%**	+0.8%
Per-layer, top-10%	1.9–2.0×	8.0/0.2	+3.7%	−0.2%

static modality already sits near the quality floor (quality loss ≈0.003), so flat→static is −88 to −90% energy for almost no quality.

A second failure: synthetic thresholds do not transfer. Uniform attention is ≈0.0011 and even the 99th-percentile visual EMA (≈0.0008) is below the synthetic promote threshold (0.006), so nothing visual is ever promoted; the policy then only demotes text, with −23 to −27% energy but +37 to +42% quality loss. Re-expressing thresholds in multiples of uniform attention (a deviation we record) and following the protocol of Sec. 4 (fit on two traces, test on the third) gave thresholds 2.0 × /0.4× and, on the held-out trace, −14.4% energy but +11.9% quality loss: the no-worse-quality criterion **failed**. **Conclusion of E2:** the M4 trade-off was a property of the synthetic trace. We kept the negative result and asked what the layer- and head-averaging was hiding. A per-layer look at one trace shows visual attention mass varying from ∼3% to 39% across layers (layer 0: 39%) and the sink being ≈0% in layers 0–1 but 40–78% in layers 12–30, so one averaged entry per token plausibly hid the structure a policy could use.

5.3 E3 — Per-(token, layer) entries, sparse reads, and measured INT4 error

Changes. (i) One entry per (token, layer), with each layer's attention renormalised so the step's total mass stays 1; (ii) a *sparse-read* model (top-k% entries by live importance plus the newest entries; skipped mass is reported and costs nothing; the importance estimate is an upper bound, oracle and free); (iii) a measured INT4 error (Sec. 3.2). Same protocol as E2.

Finding: the first real-data win. With per-layer entries and dense reads the dynamic policy saves ∼20% energy on all three traces (−20.6/−19.4/−20.0%) at roughly neutral quality (−2.0/−10.3/+0.8%), using 1.2–1.4k tier moves per session (Tab. 2). The held-out quality (+0.8%) technically misses the strict "no worse than static" rule, which we state rather than round. Two boundary results appear: sparse reads shrink the headroom from ∼ 9.3× to ∼ 1.6–2× and the dynamic policy either has nothing to do (layer-average) or loses (per-layer, +3.7 to +7.5%): with sparse reads HOT text entries are rarely read, so demoting them saves little while the moves still cost.

What this left open. (a) An explanation: our working hypothesis was a *layer effect*. (b) $n = 3$ images, one model, one held-out trace.

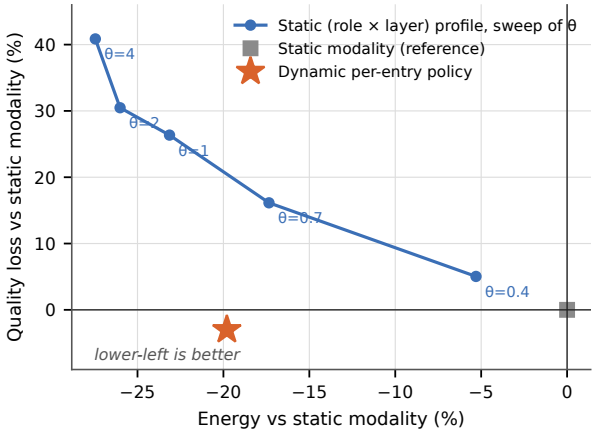


Figure 5: E4: the static (role × layer) profile traces a trade-off curve (dense reads, mean of three held-out folds) and never reaches the dynamic policy. At matched energy ($\approx -20\%$) the static profile loses roughly 16–26% in quality while the dynamic policy gains 3%.

Table 3: E5, dense reads, 10× reconfiguration cost, leave-one-trace-out over 42 images. Percentages are vs static modality, per trace, then averaged; brackets are paired-bootstrap 95% CIs. Quality loss < 0 is better.

Setting	Energy	Quality loss	Selected $\theta_\uparrow/\theta_\downarrow$	Passes criterion
256M, leave-one-out	-22.3% $[-22.8, -21.9]$	-2.5% $[-4.0, -1.1]$	8/0.4 (42/42 folds)	yes
500M, leave-one-out	-23.4% $[-24.1, -22.6]$	-1.8% $[-4.6, +0.9]$	4/0.8 (42/42)	yes
256M thresholds → 500M	-23.6% $[-24.1, -23.1]$	$+0.4\%$ $[-1.4, +2.1]$	8/0.4 (no refit)	yes
256M, top-10% reads	$+5.7\%$ $[+5.4, +5.9]$	-0.7%	32/0.02	**no**
500M, top-10% reads	$+5.7\%$ $[+5.4, +6.0]$	-0.6%	32/0.02	**no**

5.4 E4 — Is the per-layer win merely a layer prior? A falsification test

Hypothesis (stated before testing). If the win is a layer effect, a *static* profile, a group importance map by (modality, layer) or (role, layer) with role $\in$ {sink, prompt text, generated, visual} built offline and with zero moves, should capture it. **Protocol.** Leave-one-trace-out profile and threshold θ (eight values from 0.05 to 4.0), plus an in-sample ceiling.

Finding: hypothesis falsified. At matched energy, the static profile has clearly worse quality than the dynamic policy (Fig. 5). The (modality, layer) grouping is degenerate (for $\theta \geq 0.4$ it selects exactly the text groups, i.e. static modality), and the in-sample ceiling (+27.2% energy, −18.4% quality) nearly equals the cross-validated result (+27.7%, −18.6%), so this is neither under-fitting nor leakage: layer- and role-level information is too coarse. **Correction.** We had written that the saving came from "demoting text entries in layers where text is unimportant". The arithmetic stands (text entries are ∼45% of static energy and the saving is about half of that) but the *layer* part was wrong: no text layer group is unimportant on average. The saving comes from variation among individual text entries, i.e. which prompt tokens matter in this prompt, which only a per-entry, online signal sees. **What this left open:** everything rests on three images and one model.

5.5 E5 — Scale-up: 42 images, three image kinds, two model sizes

Pre-specified claims (fixed before running; Sec. 4): (a) the dense-read win holds under leave-one-trace-out; (b) its dependence on the reconfiguration cost; (c) its dependence on sparse read fraction; (d) transfer to a larger model (256M thresholds applied unchanged to 500M). Min-dwell variants were an additional check on move count.

(a) The win is robust for dense reads. The saving is negative on every one of the 42 traces. By image kind (256M): charts −22.8%, documents −20.2%, natural photos −22.9%. Quality is *neutral, not improved*: the per-trace rate of "energy down and quality no worse" is 76% on 256M and 55% on 500M, and the 500M quality interval includes zero. The ladder of Fig. 6 shows how the estimate moved from E2 to here.

Grid edge. The first 256M pass selected the grid corner (promote 8×, demote 0.4×) in all 42 folds, which would make leave-one-out meaningless and suggested the optimum might lie outside the grid. We widened the grid (promote up to 32×, demote up to 1.6×) after seeing this and re-ran (first-pass results

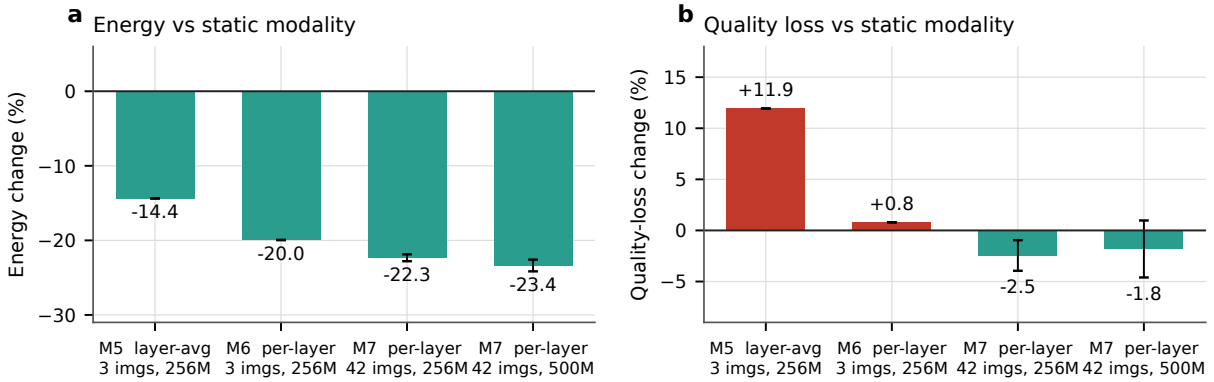


Figure 6: Held-out effect of the dynamic policy across the stages of the study. The first bar is E2 (single entry per token), the second E3 (per layer); the last two are E5. Error bars are 95% CIs (not available for the 3-trace stages).

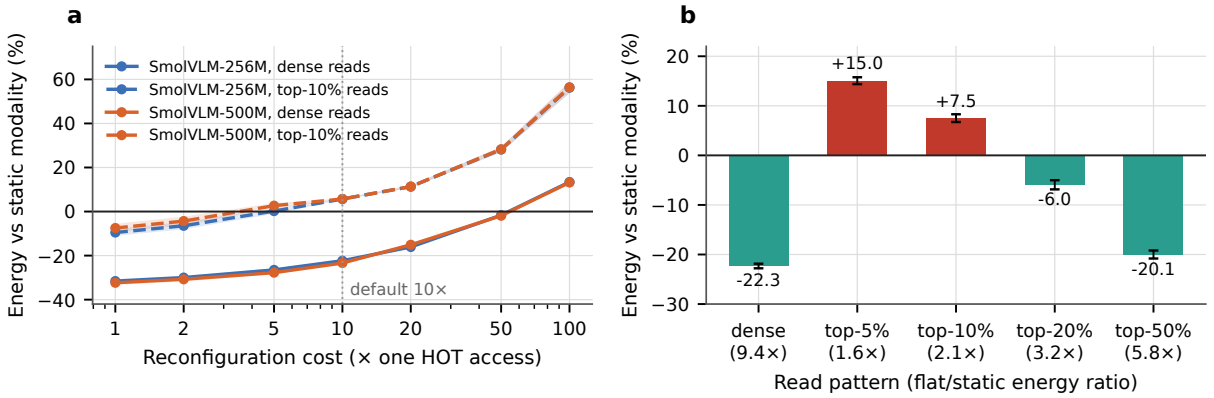


Figure 7: (a) Energy saving vs reconfiguration cost (thresholds re-selected per cost; bands are 95% CIs); the saving breaks even near 50×. (b) Dense reads are required: the saving shrinks and reverses as reads become sparse (flat/static energy ratio in parentheses).

are kept). The 10× dense result *did not change* (still −22.3%, 8/0.4 in all folds): the corner was a real optimum and not a search limit.

(b) Reconfiguration cost. Re-selecting thresholds at each cost, the saving is −31.6% at 1×, −26.5% at 5×, −16.1% at 20×, −1.7% at 50× and +13.4% at 100× on 256M (500M: −32.3, −27.7, −15.1, −1.9, +13.2). The break-even is about 50× on both models, leaving a factor of about five of margin at the default 10× (Fig. 7a). **(c) Sparse reads.** With top-10% reads the dynamic policy *loses* (+5.7% energy on both models, every trace worse). The 256M sweep over read fraction gives −20.1% (top-50%), −6.0% (top-20%), +7.5% (top-10%) and +15.0% (top-5%) (Fig. 7b): the advantage needs dense or nearly dense reads. The chosen thresholds at 10% (32/0.02) sit at a new grid corner, so we cannot exclude a modest shift with a wider grid, but not a change of sign of this magnitude. **(d) Transfer.** 256M thresholds applied unchanged to 500M give −23.6% energy at +0.4% quality, essentially the same as 500M's own cross-validated optimum.

Mechanism. Ablating the policy at the selected 256M thresholds shows where the saving comes from (Fig. 8): demote-only gives −24.2% energy but *worsens* quality by +6.8%; promote-only costs +1.8% energy and improves quality by −7.6%; the full policy gives −22.3% energy and −0.8% quality. **Demotion earns the energy; promotion buys back the quality it would otherwise give up.** 87% of moves (1008 of 1156 per session) are on prompt-text entries, consistent with E4. A min-dwell requirement only trades saving for fewer moves (0/5/20/40 steps: −22.3/ − 19.8/ − 13.8/ − 5.9%), so the default no-dwell policy is preferred.

What E5 left open. Two objections that no amount of further scaling answers: (1) the policy consumed the *true* attention of every entry for free; (2) "quality" is an attention-weighted error proxy, not a measurement of what the model says. The next two experiments address them.

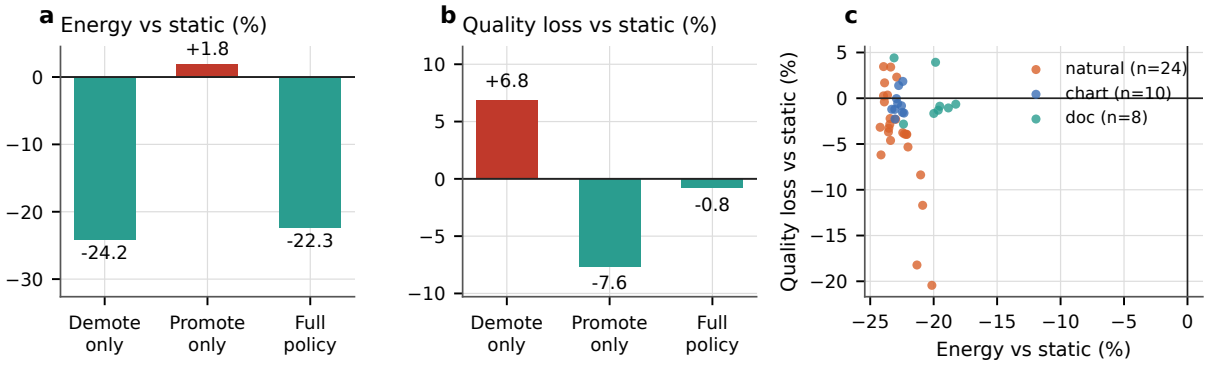


Figure 8: Mechanism ablation on 256M at the selected thresholds (a, b), and the per-trace outcome of the full policy by image kind (c): all 42 points have negative energy change; quality scatters around zero.

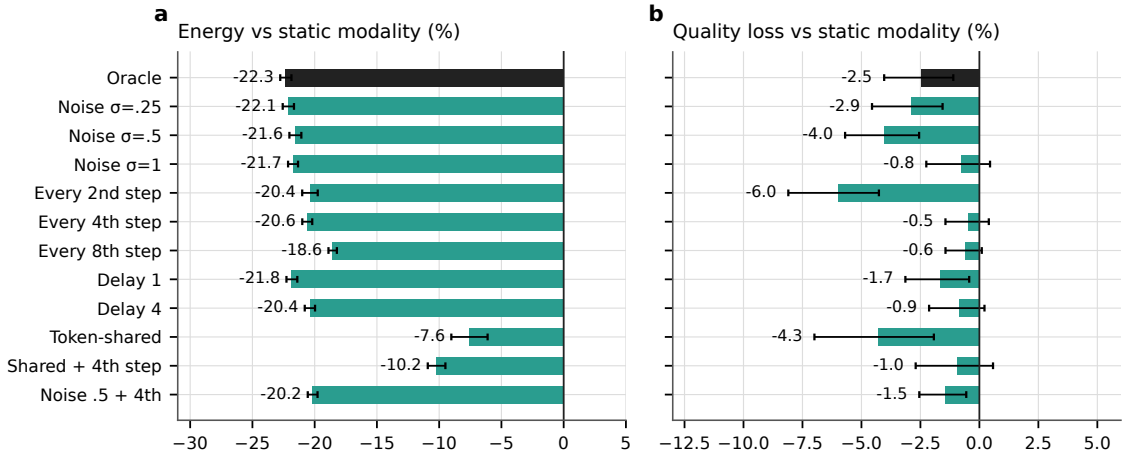


Figure 9: E6: held-out effect of the dynamic policy under imperfect importance signals (256M, 42 traces, dense reads, 10×; bars are leave-one-out means with bootstrap 95% CIs; black = oracle). Per-entry signals can be noisy, sparse in time or stale and keep ≥18.6% of the saving; a single score per token does not.

5.6 E6 — Objection 1: the policy used the true attention for free

Question. E3–E5 fed the importance monitor the exact attention weight of every (token, layer) entry at every step. A real system must compute, approximate or sample this signal. How much of the saving survives an imperfect one? **Setup.** An *estimator* sits between the true weights and the monitor (Fig. 2); quality and access energy are still charged with the *true* weights, so the policy simply acts on a worse signal. We test seven degradations on the 42 256M traces with dense reads, 10× cost and the same leave-one-out protocol (thresholds re-selected per estimator over a 10-configuration grid; the oracle row reproduces E5):

- *Noise* $\sigma \in \{0.25, 0.5, 1\}$: each weight is multiplied by an independent log-normal factor (for σ=1, a factor within $e^{\pm 1}$ for 68% of reads and mean-inflated by 1.65×).
- *Sampled*: the monitor sees attention only every 2nd, 4th or 8th step (EMA decay raised to the same power to keep its time constant).
- *Stale*: the monitor sees the weights from 1 or 4 steps ago.
- *Token-shared*: one score per token, the mean over layers, shared by all of its entries, i.e. what a system obtains if it can afford only one importance value per token rather than one per (token, layer).
- Two combinations (token-shared with 4-step sampling; noise σ=0.5 with 4-step sampling).

Finding 1: the saving is robust to a degraded per-entry signal. Noise up to σ=1 leaves −21.8% energy (oracle −22.3%); observing attention only every 8th step gives −18.6%, a signal delayed by 4 steps gives −20.4%, and noise plus 4-step sampling gives −20.2% (Fig. 9). All of these meet the pre-specified criterion (energy saving ≥ 10%, quality change ≤ +2%, CI excluding 0). Quality does not degrade: it stays at or below the static baseline (between −6.0% and −0.5%), because a noisier or older signal promotes slightly more entries. The number of moves is essentially unchanged (1111–1298 against 1190 for the oracle).

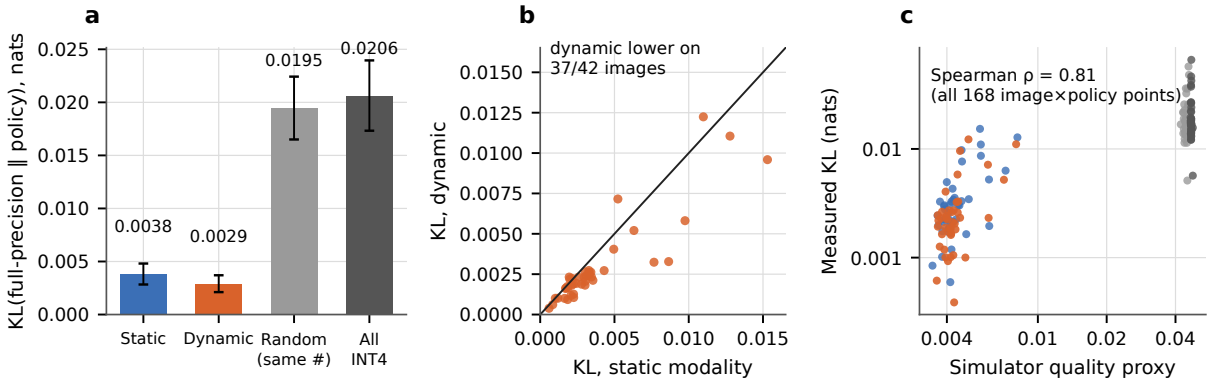


Figure 10: E7: real output change under INT4 placement, 42 images, SmolVLM-256M. (a) Mean KL to the full-precision model (95% CIs). (b) Dynamic vs static per image: points below the diagonal favour the dynamic policy. (c) The simulator's quality proxy against measured KL for all 168 (image, policy) points.

Table 4: E7: measured effect of INT4 placement on the next-token distribution (mean over 42 images). Fraction of entries in INT4 is the mean over steps.

Placement	INT4 entries	KL (nats)	Top-1 agreement	Proxy ϵmass
Static modality	91.8%	0.00382	0.976	0.0045
Dynamic (ours)	94.9%	**0.00291**	0.977	0.0044
Random, same INT4 count	94.9%	0.01946	0.930	0.0445
All INT4	100%	0.02064	0.929	0.0468

Because the saving equals the energy a system may spend on estimation, the oracle's 22.3% of static access energy is also the upper bound on the estimator's own cost per session, and a signal sampled every 4th step still leaves ∼20% of headroom.

Finding 2: the signal must stay per (token, layer). Collapsing importance to one score per token cuts the saving to -7.6% $[-9.0, -6.1]$ (with 4-step sampling, -10.2%), below the 10% criterion. At the oracle's thresholds the token-shared signal reaches -18.7% energy but loses $+12.8\%$ quality, so the policy cannot find an operating point as good as the per-entry one. This is the same lesson as E4, seen from the signal side: what the policy exploits is variation *between layers of the same token*, not a per-token summary. **Caveat.** All estimators here are free; we charge no energy or latency for producing them, and do not model any specific approximate scorer (e.g. low-precision or sampled key scores). E6 shows how imperfect the signal may be, not what a concrete estimator costs.

5.7 E7 — Objection 2: is the quality proxy real?

Question. Everything so far scores quality with $\epsilon\times$(attention mass served from WARM). Does a policy that the proxy ranks better also change the real model's output less when INT4 is actually applied? **Setup.** For each of the 42 images, we replay each policy's placement on the real SmolVLM-256M. The simulator logs, for every decode step, which (token, layer) entries are in WARM; the model is then teacher-forced on its own full-precision 64-token generation, and at step t the cached K and V of exactly those entries are replaced by their INT4 round trip (per-token asymmetric min-max, groups of 32). We measure the KL divergence $\mathrm{KL}(p_{\text{full}} \,\|\, p_{\text{policy}})$ of the next-token distribution (mean over the 64 steps) and top-1 agreement with the full-precision model. The replay reproduces the saved generation on all 42 images. Four placements are compared: static modality, the dynamic policy (selected thresholds 8/0.4), *random placement with the same number of INT4 entries at every step as the dynamic policy* (a control for *which* entries are demoted), and everything in INT4.

Finding 1: the policy picks the right entries. The dynamic policy places *more* entries in INT4 than static modality (94.9% vs 91.8%) and still changes the output distribution **21.8% less** (per-image mean, 95% CI $[15.3, 28.1]$%; lower on 37 of 42 images: 22/24 natural, 9/10 chart, 6/8 document). Random placement with the same INT4 count is **9.4**$\times$ worse than the dynamic policy ($[7.8, 11.1]\times$; worse on all 42 images) and indistinguishable from putting everything in INT4. The energy saving of E5 is therefore not obtained by demoting entries at random, and the entries the policy keeps in HOT are the ones the real

model is sensitive to.

Finding 2: the proxy is a usable but conservative guide. Across all 168 points the proxy and the measured KL are strongly rank-correlated (Spearman ρ=0.81; Pearson on log scale 0.88), mostly because the proxy separates policies that demote important entries from those that do not (Fig. 10c). Within the static/dynamic pair the correlation is weaker (ρ=0.49), the proxy's per-image sign of the dynamic-vs-static difference matches the measured one on 29 of 42 images, and it *understates* the advantage: the proxy predicts −2.5%, the measurement −21.8%. Top-1 agreement does not separate dynamic from static (difference +0.1 points, CI includes 0). **Caveats.** The effects of INT4 on this small model are small in absolute terms (even all-INT4 keeps 93% top-1 agreement over 64 teacher-forced steps), so E7 validates the *ordering and mechanism* rather than a task-accuracy claim, covers one model and the single INT4 scheme measured in Sec. 3.2, and uses KL as the quality measure, not a benchmark score.

6 Discussion

What is established. For dense per-layer reads, under one stylised but fixed cost model, importance-driven placement beats the strong static-modality baseline by ∼22–23% energy at neutral quality on two model sizes and three image kinds, with thresholds that transfer across sizes and a break-even reconfiguration cost about five times the default. The saving is produced by demoting individual prompt-text entries whose importance is low *in this prompt*, and the promotions that give quality back are few. The two added checks narrow the main objections: the gain does not require an oracle (it survives noisy, stale or sampled importance, provided importance stays per (token, layer)), and it is not an artefact of the quality proxy (when INT4 is applied in the real model, the policy keeps in HOT the entries that matter: its output divergence is 21.8% below static modality's and 9.4× below random placement's). They do not replace hardware measurement, and neither the estimator's own cost nor a concrete approximate scorer is modelled.

Where it does not apply. (i) Sparse reads: with top-10% reads the policy is a net loss and the HOT/WARM headroom itself shrinks to ∼2×. (ii) Reconfiguration costs above roughly 50× a HOT access. (iii) Coarse-grained placement: a static layer or role map cannot reproduce the effect. These boundaries are as much a part of the finding as the saving.

Why the negative results stayed in. E2 and E4 overturned beliefs we held (that the synthetic trade-off was general; that the win was a layer effect). Keeping them is what makes the later claim credible: the scale-up in E5 was designed to be able to fail, and the criterion was fixed beforehand.

Limitations.

- **Stylised cost model.** WARM is 10× cheaper per byte and 1.5× slower than HOT; the reconfiguration multiplier is swept but these ratios are not. Results are ratios between policies under shared assumptions, not device energies. There is no hardware measurement.
- **Models and data.** Two sizes of one model family, 42 images, 64 decode steps, one prompt per image. Larger models (we planned SmolVLM-2.2B and dropped it), other families and long-form generation are untested. Attention is head-averaged per layer; per-head entries are not modelled.
- **Two tiers, unlimited capacity.** No capacity pressure on HOT, no bandwidth contention, serialised latency.
- **Scope of the two added checks.** E6 charges nothing for computing the importance signal. E7 covers one model, one INT4 scheme (the one measured in Sec. 3.2) and measures distributional change (KL over 64 teacher-forced steps), not benchmark accuracy; INT4 effects on this small model are small in absolute terms.
- **Post-hoc choices** (logged above): the damping rule in E1, the threshold normalisation in E2, the grid widening in E5.

7 Conclusion

We asked whether importance-aware tiering of multimodal KV caches pays off, and answered with a chain of experiments in which every step either confirmed or overturned the last. The answer is conditional: yes

for dense reads, per-entry importance and moderate reconfiguration cost, by ∼22–23% energy at neutral quality, robustly across 42 images and two model sizes; no for sparse reads or very costly moves. The study also leaves a usable method: fix the selection rule and pass criterion before looking, report held-out numbers with intervals, and keep the experiments that failed.

Reproducibility. All code, traces, per-simulation CSVs and this paper's figure scripts are in the repository. `experiments/run_m7.py`, `analyze_m7.py`, `run_m7_estimators.py`, `validate_real_output.py` and `paper/make_figures.py` regenerate Sections 5.5–5.7; the lab notebook `NOTES.md` records every deviation chronologically.

References

[1] Tu, D., et al.: VL-Cache: Sparsity and modality-aware KV cache compression for vision-language model inference acceleration. arXiv (2024).

[2] He, E., et al.: ZipVL: Efficient large vision-language models with dynamic token sparsification. arXiv (2024).

[3] Huang, K., et al.: AirCache: Activating inter-modal relevancy KV cache compression for efficient large vision-language model inference. arXiv (2025).

[4] Yang, C., et al.: TopV: Compatible token pruning with inference time optimization for fast and low-memory multimodal vision language model. CVPR (2025).

[5] Chen, L., et al.: An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. ECCV (2024).

[6] Zhang, Z., et al.: H2O: Heavy-hitter oracle for efficient generative inference of large language models. NeurIPS (2023).

[7] Li, Y., et al.: SnapKV: LLM knows what you are looking for before generation. NeurIPS (2024).

[8] Xiao, G., et al.: Efficient streaming language models with attention sinks. ICLR (2024).

[9] Liu, Z., et al.: KIVI: A tuning-free asymmetric 2bit quantization for KV cache. ICML (2024).

[10] Tang, J., et al.: Quest: Query-aware sparsity for efficient long-context LLM inference. ICML (2024).

[11] Kwon, W., et al.: Efficient memory management for large language model serving with PagedAttention. SOSP (2023).

[12] Marafioti, A., et al.: SmolVLM: Redefining small and efficient multimodal models. arXiv (2025).

[13] Lin, T.-Y., et al.: Microsoft COCO: Common objects in context. ECCV (2014).